

Experiences with Agile Practices in the Global Studio Project

Roger Urdangarin, Paulo Fernandes
PUCRS-FACIN
Av. Ipiranga, 6681
Porto Alegre, RS 90.619-900, Brazil
urdangarin@inf.pucrs.br

Alberto Avritzer, Daniel Paulish
Siemens Corporate Research
755 College Road East
Princeton, NJ 08540, USA
alberto.avritzer@siemens.com

Abstract

We report on our experiences using agile practices in a global software development project. Specifically, we report on the communication and collaboration patterns that were discovered using the social network analysis methodology. We used as a case study the Global Studio Project Version 3.0, where Extreme Programming practices were applied to one of the remote software development teams. We summarize our conclusions about Extreme Programming usage in global software development projects by presenting ten lessons learned from the application of Extreme Programming practices to the Global Studio Project Version 3.0.

1. Introduction

The Global Studio Project (GSP) is a student-based software development project that has been instrumented for data collection to support empirical studies on communication, coordination and collaboration among distributed teams. The student-based development teams that compose GSP simulate an industrial global software development project and use collaboration tools for communication among distributed sites. The product that was developed by the student teams for GSP V3.0 is a performance analysis tool set that is used to simulate, model, and analyze the performance of large scale industrial software systems.

Today's software project managers have a large number of possible ways to structure their Global Software Development (GSD) project across multiple development sites. If a software architecture design exists at the time when the development work is being allocated among development sites, it will often be a driver for the project's organizational structure. Some of the project organizational approaches that could be considered include:

- **Product Structure:** The architecture decomposes the system into components and the components are allo-

cated as work packages to the different sites.

- **Process Steps:** Work is allocated across the sites in accordance with the phases of the software development process; e.g., design may be done at one site, development at another site, and testing at yet another site.
- **Release:** The first product release is developed at one site, the second at another site, etc. Often, the releases will be overlapped to meet time-to-market goals; e.g., one site is testing the next release, another site is developing a later release, and yet another site is defining or designing an even later release.
- **Platform:** One site may be developing reusable core assets of the product line and other sites may be developing application-level software that uses the platform.
- **Competence Center:** Work is allocated to sites depending on the technical or domain expertise located at a site. For example, perhaps all user interface design is done at a site where usability engineering experts are located.
- **Open Source:** In an open source structure, many independent contributors develop the software product in accordance with a technical integration strategy. Centralized control is minimal except when an independent contributor integrates his code into the product line.

These organizational approaches may change over time. For example, components may be allocated at first with the intent that the remote site will develop the skills over time to become a competence center in the functionality that component provides.

In addition to the organizational structure, a global software development process must be selected or created which supports the structure. During the first two years of the GSP, a product structure approach was used to structure the organization and an "extended workbench model"

development process was used [18]. This resulted in a hub and spoke kind of organizational structure where the remote component development teams communicated mostly with the central team roles (e.g., chief architect, project manager, supplier manager) at the headquarters or central site.

For GSP V3.0, more of an open source approach was used, with competence centers located at the remote sites, since each university had expertise on the specific performance analysis tool that they had developed which was to be integrated into the performance analysis tool set. The tools had already been individually developed and had to be integrated using a “system of systems” type of process where a central team was used for integration but had much less control than with the extended workbench model.

The remainder of this paper is organized as follows: In Section 2, we present a brief overview of the related literature. In Section 3, we present the methodology we used for data collection from the case study presented in this paper. In Section 4, we present the lessons learned from the case study. In Section 5, we present our conclusions and suggestions for further research.

2 Related Literature

Experiences with the Global Studio Project (GSP) have been reported in a number of papers, and it has been documented as a case study (GSP 2005) within [18]. Damian [5], Herbsleb et al. [10], Bradner and Mark [3] have identified communication and cultural differences as the most common non-technical barriers that are usually encountered in global software development projects. In addition, technical factors such as network infrastructure, software development environment heterogeneity, and the selection of the specific site where these activities take place, were also identified as significant barriers to the successful implementation of global software development projects.

The selection of a global software development process methodology for the development and testing of a software project introduces new challenges to the program management of such efforts. Project managers of global software development projects must address issues related to several timezones, large geographical distances, conflicts generated by lack of cultural sensitivity among team members, lack of frequent communications, and the resulting lack of trust among members of remote teams [12] [8].

Damian et al. [6] raised the issue of the more formal nature of the communications among remote team members, as contrasted to the more informal nature of communications within a co-located team. They concluded that there exists an increased likelihood of the establishment of strong personal relationships within a co-located team that may lead to more opportunities for communication among team members that are co-located and for the informal ex-

change of knowledge related to the project. Therefore, co-located projects benefit from the informal exchange of information, while globally distributed software development projects are likely to encounter challenges in its communication structure.

In [3, 6, 4], studies of the impact of geographical distances on coordination and control of global software development projects are presented. Paulish et al. [16], recommend that remote software development teams have face-to-face interactions to boost the levels of communications. Herbsleb [11] reported that geographical distances impact the level of informal communications among remote teams. In [11], it was recommended that team members should be able to identify and contact the correct person to address questions and issues. Therefore, it is very important to prevent the situation where all communication to a specific remote team flows through a designated team member, as this designated team interface may become a communication bottleneck. Layman et al. [12] report that several global software development challenges could be overcome by increased informal communication among remote team members.

We show in Table 1 a comparison of several case studies that use agile methodologies to boost the levels of informal communication among remote team members.

3 Research Methodology

In this section, we describe the research methodology used in this paper. The objective of this study was to investigate the risks and benefits of using *Extreme Programming* methodologies in one of the remote teams engaged in a global software development project. The Global Studio Project Version 3.0 (GSP V3.0) was used as the test bed for data collection. GSP V3.0 was composed of a central team located at Siemens Corporate Research, in Princeton, USA, and three remote teams, located at COPPE/UFRJ, Rio de Janeiro, Brazil, PUCRS, Porto Alegre, Brazil, and L’Aquila University, L’Aquila, Italy. We present in Figure 1 an architecture diagram showing the modules developed in GSP V3.0 and the scope of responsibility of each remote team.

3.1. Data Collection

The data collection approach used in this paper was composed of two questionnaires. The first questionnaire was used to collect data related to the social network analysis (SNA) interactions among the GSP V3.0 team members. This questionnaire was used by each team member, every two weeks, to answer questions related to frequency and importance of the communication with other team members. This questionnaire was used to gather quantitative

Table 1. GSD and Agile Methods Literature Summary

Authors	Research Method	Subjects	Agile practices Validated	GSD Challenges Emphasized
Damian, Williams, Layman and Bures [7]	Case Study	Industrial Team	Unknown/Unspecified	Communication
Grossman, Bergin, Leip Merritt and Gotel [9]	Case Study	Industrial Team	Unknown/Unspecified	Communication
Layman, Williams and Cunningham [13]	Case Study	Industrial Team	Pair Programming Refactoring Simple Design Collective Ownership Continuous Integration Coding Standards Sustainable Pace Metaphor	Communication
Paasivaara and Lassenius [15]	Case Study	Students	Unknown/Unspecified	Communication Collaboration
Ramesh, Cao, Mohan and Xu [17]	Case Study	Industrial Team	Unknown/Unspecified	Communication Control Trust
Xiaohu, Bin Zhijun and Maddineni [20]	Case Study	Industrial Team	Planning Game Refactoring Simple Design Continuous Integration Coding Standards Pair Programming Metaphor	Communication
This Study	Case Study	Students	Pair Programming Code Standards Collective Ownership Planning Game Test-Driven Development	Communication Collaboration

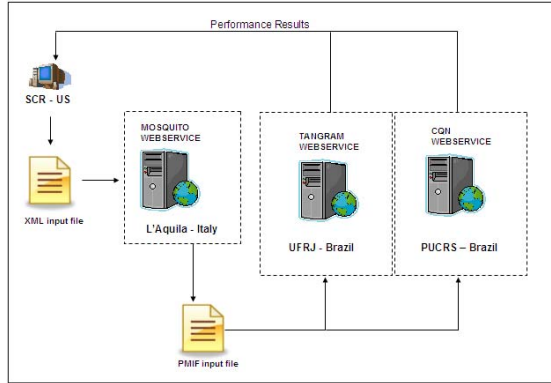


Figure 1. Modules and Scope of Responsibility for Global Studio Project V3.0

data related to social network interactions among the members of GSP V3.0. It was implemented *on-line* and was composed of ten questions designed to identify the project social network (who interacts with whom), the type of communication (technical or social), frequency of communication (weekly, daily, several times per day), and importance of the communication (how relevant was the interaction to your work). In addition, some of the questions probed social interactions outside of work. The questionnaire was web-based and the answers were stored in a relational database.

The following questions were selected from the on-line questionnaire for analysis in this paper:

- *Question A* - Over the last two weeks, what was the frequency of your communication with the following team members concerning GSP V3.0?
- *Question B*- How important was the communication in allowing you to get your work done ?

The second questionnaire was used to collect data related to the experience of one of the remote teams with the use of *Extreme Programming* methodologies in a global software development environment. This questionnaire was composed of forty-two questions that were used to help with the empirical assessment of the benefits of employing agile methods in our global software development project.

3.2. Data Analysis Methodology

The social network graphs presented in Section 4 are based on the social network analysis questionnaire described in the previous subsection. These graphs use narrow and bold edges to represent low or high frequency of interaction and low or high importance of interaction. For Question A, frequency of communication, the following convention was used:

Table 2. Team Members for GSP V3.0.

Country	Site	Member Initials/Role
USA	SCR	AA/Project Manager AB/Researcher
Italy	L'Aquila	VC/Researcher
Brazil	UFRJ	FD/Lead Architect GF/Developer
Brazil	PUCRS	EM/Developer EN/Team Leader FF/Team Leader LM/Developer MC/Developer PF/Researcher RU/Test Leader VT/Developer

- Narrow edges represent a weak communication link as represented by the following answers to the communication frequency question: *once in two weeks* or *at most once a week*.
- Bold edges represent a strong communication link as represented by the following answers to the communication frequency question: *more than once a week* or *at least once a day*.

For Question B, importance of communication, the following convention was used:

- Narrow edges represent a weak importance link as represented by the following answers to the importance of communication question: *slightly important* or *moderately important*.
- Bold edges represent a strong importance link as represented by the following answers to the importance of communication question: *important* or *very important*.

Table 2 identifies the GSP V3.0 team members showing, for each team member, the country, site name, team member initials, and team member role. The team member initials from Table 2 are used in the social network analysis graphs.

4 Lessons Learned

In this section we present the social network analysis results and ten lessons learned that were derived from the social network analysis graphs.

4.1 Lesson 1: A tool specifically designed to support agile methodologies can help ensure the development team's adherence to the agile process for software development

Figure 2 shows an example of the use of *XPlanner*, a tool specifically designed to support *Extreme Programming* practices. *XPlanner* was used to collect project management data such as the number of iterations, the user history and the number of hours expended in specific tasks by each team member. This data was entered into the *XPlanner* tool by each member of the pair-programming team, therefore helping the team learn about the methodology. We conclude that the use of the *XPlanner* tool provided visibility to project management about task progress and helped ensure the team's adherence to the agile process for software development.

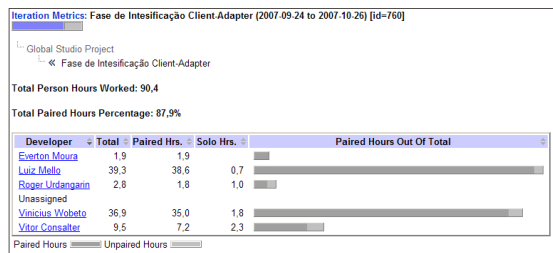


Figure 2. Sample data from the XPlanner tool, showing number of hours used by the pair-programming team

4.2 Lesson 2: The correct deployment of Extreme Programming methodologies requires that an Extreme Programming coach be made available to answer the development team's questions

Marchesi et al. [14] and Ebert et al. [8] suggest that an experienced coach be made available to answer developer questions. In the GSP V2.0, we observed the need for such a coach, as the development team struggled to use agile methodologies without proper guidance. In GSP V3.0 we successfully used a coach to answer the development team questions.

4.3 Lesson 3: Remote teams that use Extreme Programming methodologies use a more direct and frequent communication style

The PUCRS team was strongly encouraged to communicate with the remote teams using e-mail, Chat, and Skype. Figure 3 shows the communication pattern for GSP V3.0. The edges in bold in the Figure 3 are from nodes EN, PF and RU, which used the *Extreme Programming* methodology, to the nodes AA, FD and VC, which are located in nodes that are remote. We observed in the GSP V3.0 case study that the use of *Extreme Programming* methodologies by the PUCRS team, helped the team communicate better with the other remote teams participating in GSP V3.0.

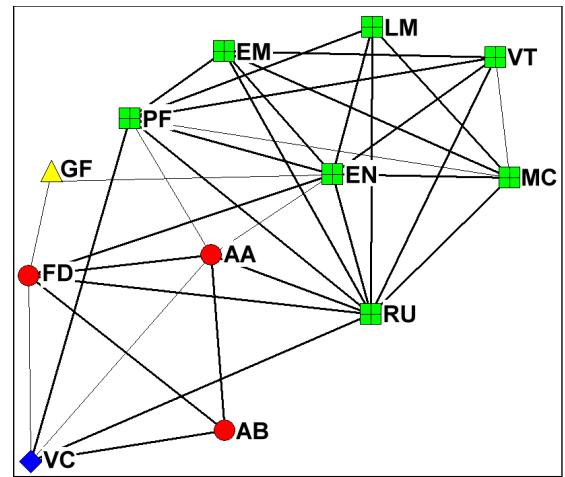


Figure 3. GSP V3.0 communication frequency among team members during second bi-weekly data collection period in May

4.4 Lesson 4: Cultural ambassadors can help overcome cultural barriers among remote teams.

Figure 4 shows the communication frequency between the remote team located at PUCRS (nodes PF and RU) with members of the central team (nodes AA and FD), which acted as cultural ambassadors for the PUCRS team.

We have identified the following benefits of having cultural ambassadors for the PUCRS team at the central site:

- Whenever problems occurred in the remote team, frequent informal communication would be initiated by members of the remote team located at PUCRS with the cultural ambassadors for PUCRS located at the central team,

- Language was never a barrier for the PUCRS team, as the team could communicate with the cultural ambassadors in their native language,
- Cultural conflicts were minimized to a certain degree, as the cultural ambassadors understood well the culture of the PUCRS team,
- The members of the remote team were committed to project success and felt a sense of partnership with the central team.

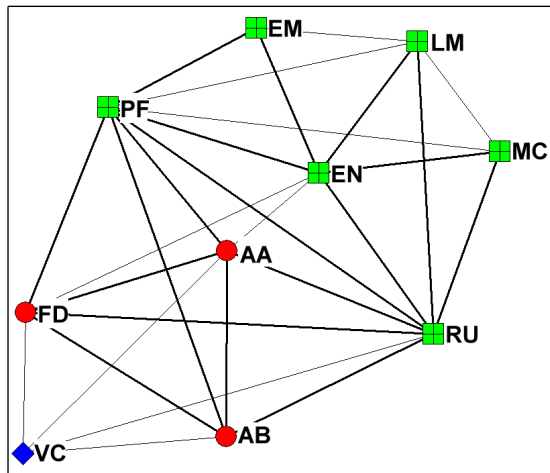


Figure 4. GSP V3.0 communication frequency among team members during first by-weekly data collection period in April

4.5 Lesson 5: The use of Extreme Programming methodologies can help increase trust among the remote teams.

Figure 5 shows in the bold edges of the SNA graph the importance attributed by the remote teams to the communication links established with members of the PUCRS team. The communication between members of the PUCRS team and the central team was not formally centralized in one person, as all members of the PUCRS team were allowed to interact freely with members of the central team and other remote team members. However, we have observed that the communications from the PUCRS team were through the architects and domain experts [2]. The exception is the team member RU who was the GSP V3.0 test leader. The frequent and informal communications between the PUCRS team leaders (EN, RU and PF) and the central team architects and domain experts (FD and AA) increased the trust between the PUCRS team and the central team.

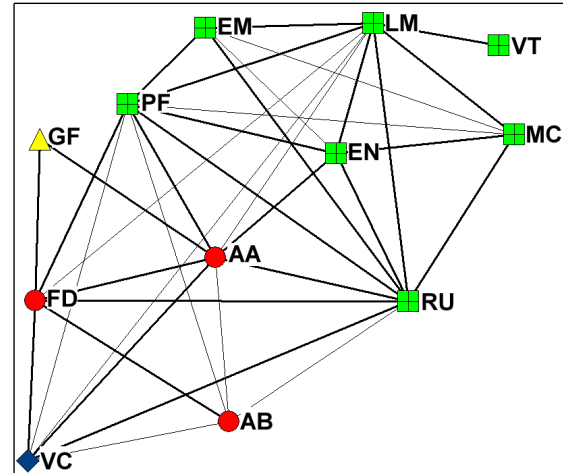


Figure 5. GSP V3.0 communication importance among team members during first by-weekly data collection period in May

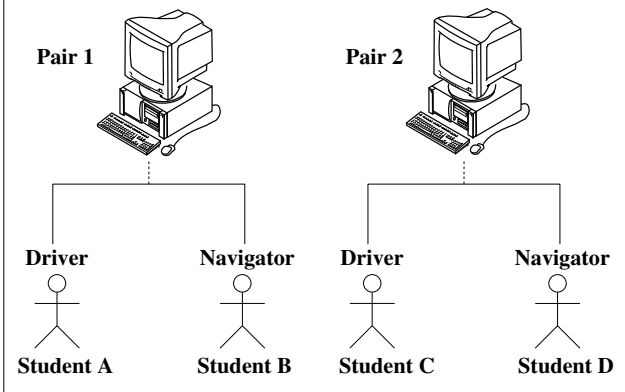
4.6 Lesson 6: The use of pair-programming practices can help with knowledge sharing among members of the local team

Figure 6 depicts graphically the methodology used to rotate tasks among peer teams. As suggested by Williams [19], task rotation was implemented for each new task. We observed that knowledge sharing among members of the local team increased after successive task rotations. This observation was corroborated by the *Extreme Programming* perception questionnaire.

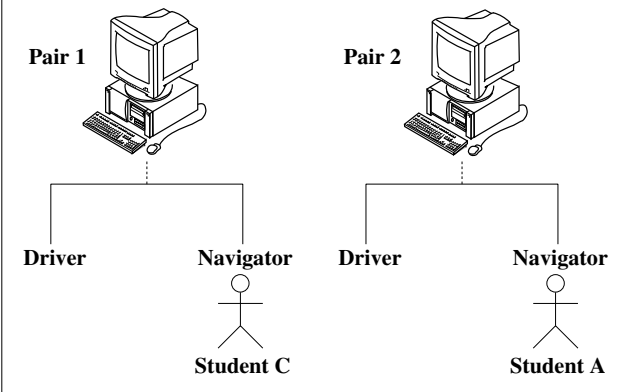
4.7 Lesson 7: The level of technical experience of the development team can positively influence the team's ability to use Extreme Programming methodologies.

Figure 7 shows the communication patterns observed for a pair-programming team composed of junior developers (EM and VT). It can be seen from Figure 7 that this pair-programming team showed frequent communication between the pair and less frequent communication to the remainder of the GSP V3.0 team members. We attributed this communication pattern to the lack of experience of the members of this specific pair-programming team, as infrequent communication of the pair-programming team can lead to the isolation of the pair-programming team from the local team.

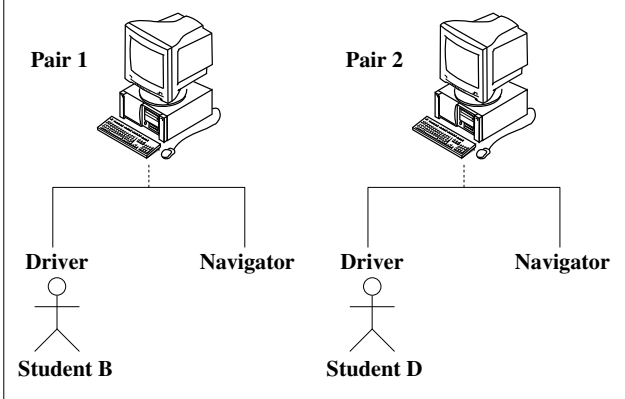
Step 1: Original pairs formation before rotation.



Step 2: Replacement of the navigator role.



Step 3: Replacement of the driver role.



Step 4: New pairs formation after rotation.

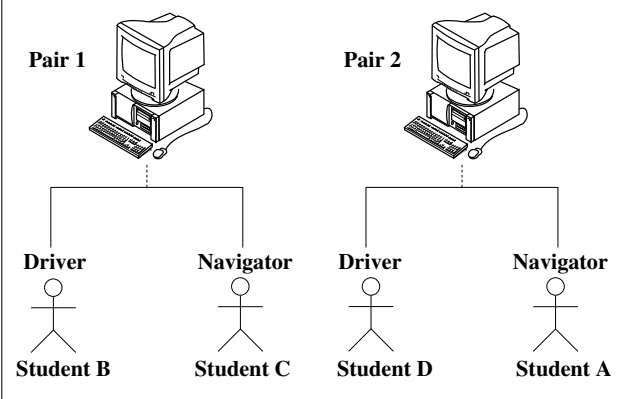


Figure 6. Pair-programming team task rotation strategy

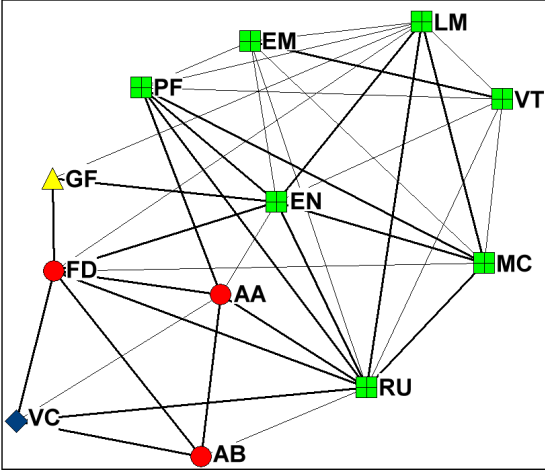


Figure 7. GSP V3.0 communication frequency among team members during second bi-weekly data collection period in May

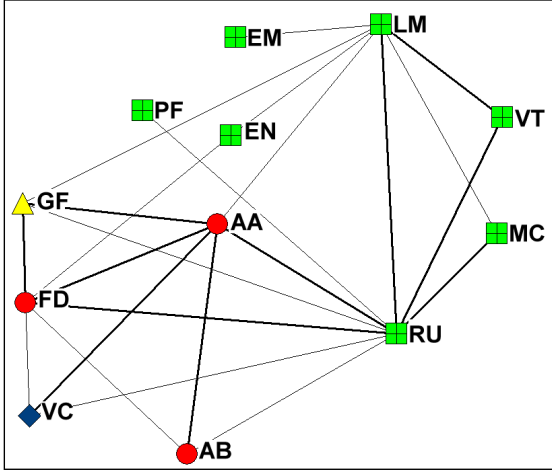


Figure 8. GSP V3.0 communication frequency among team members during first bi-weekly data collection period in June

4.8 Lesson 8: The deployment of Extreme Programming practices requires formal training of the development team in the Extreme Programming methodologies

We have learned from our case study that the junior developers had difficulty learning the *Extreme Programming* methodologies. Our developers were first year undergraduate students, and could not grasp the key concepts on their own. We therefore recommend that formal training on agile methodologies be made available to the pair-programming teams.

4.9 Lesson 9: Software development using Extreme Programming methodologies requires strong commitment of all team members

Figure 8 shows the PUCRS team communication pattern for a two-week period when the PUCRS team coach was absent. We have observed from the case study that overall team motivation was severely impacted during this period. We attribute the severe impact of the PUCRS team coach absence on productivity to the multiple roles the PUCRS team coach (EN) had in the PUCRS team, acting as team leader, developer and architect of the PUCRS component.

4.10 Lesson 10: Pro-active resource management of development staff engaged in the agile software development process is required to avoid negative impacts to the team.

Figure 9 shows the communication patterns for the PUCRS team, after the *coach* role, represented by node FF, was staffed adequately. It can be seen from Figure 9 that the communication links within the PUCRS team were restored. We have also observed that the PUCRS team productivity and motivation increased significantly after the the *coach* role was staffed.

5 Conclusions

We have presented a case study of the application of *Extreme Programming* methodologies to one of the remote teams in the Global Studio Project version 3.0. We have used social network analysis (SNA) graphs to analyze the communication patterns among the members of the Global Studio Project.

We have observed the following benefits to the project in the use of *Extreme Programming* methodologies:

- the frequent and informal communication patterns observed within the remote teams,
- the ease of knowledge transfer from more experienced staff to junior staff members,
- the encouragement the methodology provides to junior staff to engage in more complex tasks,

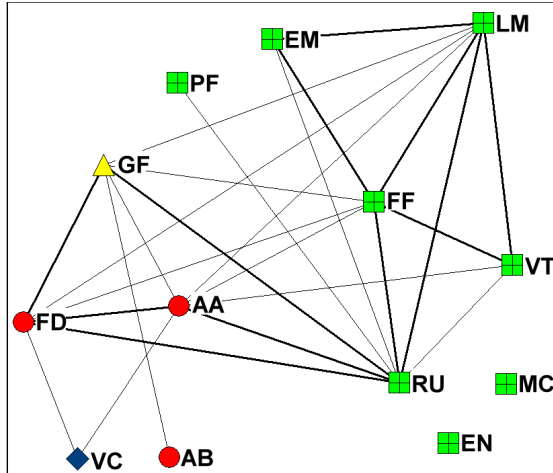


Figure 9. GSP V3.0 communication frequency among team members during second bi-weekly data collection period in June

- the increase in the developers sense of self-worth.

We have reported on ten lessons learned that were derived from the analysis of the data collected from the Global Studio Project versions 3.0. Specifically, the need for an experienced programming team and adequate training of the team members were reported. In addition, we have observed that the seeding of the central team with cultural ambassadors for the remote teams would be very positive as it would increase the frequency of informal communication between the specific remote team and the cultural ambassador.

We have also identified recommendations to project management to ensure the successful deployment of *Extreme Programming* methodologies:

- The use of a tool specifically designed to support the *Extreme Programming* methodology,
- The availability of a coach to answer team questions regarding the *Extreme Programming* methodology,
- The need for pro-active resource management of critical staff.

The experience report contained in this paper is the result of one experiment and therefore requires more validation. However, we believe that the ten lessons learned from the study are a valuable asset that should be used by researchers in global software development practice as guidance for further research.

We are continuing and extending our experiences with the Global Studio Project. In GSP V4.0 we are extending the UML-PM tool [1] developed in GSP V3.0 in the

usability domain. The approach used in GSP V4.0 takes advantage of the performance engineer's expertise in developing optimized models and of the requirements engineer's and architect's knowledge about the application specific behavior. As topics for further research we are considering architectures to enable efficient tool usage by requirements engineers and architects, and the graphical visualization of customer affecting metrics in the UML modeling domain.

References

- [1] A. Avritzer, W. Hasling, and D. Paulish. Process investigations for the global studio project. In *2nd International Conference on Global Software Engineering*, Germany, Aug 2007. IEEE Computer Society.
- [2] A. Avritzer, D. Paulish, and Y. Cai. Coordination implications of software architecture in a global software development project. In *Working IEEE/IFIP Conference on Software Architecture(WICSA) 2008*, Canada, Feb 2008. IEEE Computer Society.
- [3] E. Bradner and G. Mark. Effects of team size on participation, awareness, and technology choice in geographically distributed teams. In *36th Hawaii International Conference on System Sciences*, pages 150–160, USA, 2002. IEEE Computer Society.
- [4] E. Carmel and R. Agarwal. Tactical approaches for alleviating distance in global software development. *IEEE Software*, 18(2):22–29, Mar 2001.
- [5] D. Damian. Workshop on global software development. In *International Conference on Software Engineering (ICSE'02)*, pages 19–25, USA, May 2002. IEEE Computer Society.
- [6] D. Damian and E. J. Hargreaves. Can global software teams learn from military teamwork models? In *3rd International Workshop on Global Software Development (ICSE04)*, pages 21–23, UK, May 2004. IEEE Computer Society.
- [7] D. Damian, L. Williams, L. Layman, and H. Bures. Essential communication practices for extreme programming in a global software development team. *Information & Software Technology*, 48(9):781–794, 2006.
- [8] C. Ebert, C. H. Parro, R. Suttels, and H. Kolarczyk. Improving validation activities in a global software development. In *23rd International Conference on Software Engineering*, pages 545–554, CA, May 2001. IEEE Computer Society.
- [9] F. Grossman, J. Bergin, D. Leip, S. Merritt, and O. Gotel. One xp experience: introducing agile (xp) software development into a culture that is willing but not ready. In *2004 conference of the Centre for Advanced Studies on Collaborative research*, CA, 2004.
- [10] J. D. Herbsleb, A. Mockus, T. A. Finholt, and R. E. Grinter. Distance, dependencies, and delay in a global collaboration. In *2000 ACM conference on Computer supported cooperative work (CSCW '00)*, page pp.209, USA, Dec 2000. ACM.
- [11] J. D. Herbsleb, A. Mockus, T. A. Finholt, and R. E. Grinter. An empirical study of global software development: Distance and speed. In *23rd International Conference on*

- Software Engineering (ICSE'01)*, pages 81–90, USA, May 2001. IEEE Computer Society.
- [12] L. Layman. Changing students' perceptions: Analysis of the supplementary benefits of collaborative development. In *19th Conference on Software Engineering Education and Training*, pages 159–166, US, Apr 2006. IEEE Computer Society.
 - [13] L. Layman, L. Williams, , and L. Cunningham. Exploring extreme programming in context: An industrial case study. In *2nd Agile Development Conference (ADC '04), Salt Lake City, UT*, pages 32–41, US, June 2004. IEEE Computer Society.
 - [14] M. Marchesi, G. Succi, D. Wells, and L. Williams. *Extreme Programming Perspectives*. Addison Wesley Professional, USA, 2002.
 - [15] M. Paasivaara and C. Lassenius. Could global software development benefit from agile methods? In *IEEE First International Conference on Global Software Engineering*, pages 109–113, Brazil, Out 2006. IEEE Computer Society.
 - [16] D. J. Paulish, M. Bass, and J. D. Herbsleb. Global software development at siemens: Experience from nine projects. In *27th International Conference on Software Engineering (ICSE'05)*, pages 524–533, USA, May 2005. IEEE Computer Society.
 - [17] B. Ramesh, L. Cao, K. Mohan, and P. Xu. Can distributed software development be agile? *SPECIAL ISSUE: Flexible and distributed software processes: old petunias in new bowls?*, 49(10):41–46, Oct 2006.
 - [18] R. Sangwan, D. J. Paulish, N. Mullick, and M. Bass. *Global Software Development Handbook*. Auerbach Publications, USA, 2006.
 - [19] L. Williams and R. Kessler. *Pair Programming Illuminated*. Addison Wesley Professional, USA, 2002.
 - [20] Y. Xiaohu, X. Bin, H. Zhijun, and S. Maddineni. Extreme programming in global software development. *Electrical and Computer Engineering*, 4(2-5):1845–1848, May 2004.